Charotar University of Science and Technology

Chandubhai S. Patel Institute of Technology (CSPIT), CHARUSAT

Department of Information Technology / Computer Engineering

ITUE301: Advanced Web Development Frameworks

Open-Book Practical Examination | B.Tech. Semester 5 | AY 2026–27

Date	24 / 08 / 2026
Branch	CE / CSE / IT / AIML
Duration	2 Hours
Total Marks	20 Marks (4 Marks × 5 Tasks)
Tech Stack	React (frontend) + Express.js (backend) + MongoDB with Mongoose
Internet	Permitted for official documentation, MDN, Stack Overflow reference.
AI Tools	Permitted: ChatGPT, GitHub Copilot, Claude, Gemini, etc. may be used. Students must be able to explain every line of submitted code during viva.
Code Sharing	Strictly prohibited: identical submissions across students will be awarded zero.

General Instructions

1. Read the complete question paper before starting implementation. Plan your folder structure first.
2. Attempt all five tasks in your allotted set. Partial implementation earns partial marks: do not leave any task blank.
3. Use React for frontend tasks, Express.js for backend tasks, and MongoDB with Mongoose for database tasks. No other database is permitted.
4. GitHub repository must be public and named exactly: itue301-exam-[roll-number]-[batch] (e.g. itue301-exam-24it045-A).
5. Use a .env file for the MongoDB connection string and PORT. Do not hardcode credentials. Commit a .env.example; never commit the real .env.
6. Backend must start with node server.js or npm start. Document this in README.md.
7. AI code-generation tools are permitted for this exam (see AI Tools row above). Regardless of how the code was produced, you must be able to explain it in viva: code you cannot explain will be marked zero for that task.

8. Sharing code with another student is strictly prohibited. Copying another student's code results in a zero for both students.
9. Include a README.md with project name, frontend/backend run instructions, and MongoDB setup steps.
10. Final submission must include **both** of the following before the examination window closes: (a) your public GitHub repository link together with the final commit SHA, and (b) the PDF report named as specified in your set. Submit both via the Google Form shared by faculty. Late submissions lose 1 mark per 5 minutes beyond a 5-minute grace period.

SET E: DriveEasy Car Rental System

ITUE301 · Open-Book Practical Examination · CSPIT, CHARUSAT · 24/08/2026

Scenario: DriveEasy runs a small self-drive car rental fleet booked entirely through phone calls, causing double-bookings on the same vehicle. Build a booking platform for customers, a fleet manager, and an admin.

Data Entities:

- **Customer**: name, email, phone, licenseNumber
- **Vehicle**: model, type, pricePerDay, available (Boolean)
- **Booking**: customerId (ref), vehicleId (ref), pickupDate, returnDate, status (reserved | ongoing | completed | cancelled)

User Roles: CustomerFleet ManagerAdmin

Mandatory Requirements

- React frontend with at least three distinct pages (Login, Fleet, MyBookings).
- Client-side routing: Fleet and MyBookings must be protected routes.
- A reusable VehicleCard component that accepts props and renders vehicle details.
- Global auth state via useState / useContext or Zustand.
- Express backend exposing at least 5 REST endpoints at /api/v1/.
- All protected routes pass through a custom authGuard middleware; global requestLogger applied.
- MongoDB collections for Customer, Vehicle, and Booking with Mongoose schemas.
- Schema-level validation (required, enum, min) on all three models.
- Lazy-loaded route for an Admin Panel using React.lazy + Suspense.
- API call from React using fetch or Axios; display a loading state while data is fetched.

4 Marks

Task 1: React Component Architecture

Create the basic React frontend for the DriveEasy Car Rental System.

Create the following page/components: LoginPage, FleetPage, MyBookingsPage, VehicleCard.

Place reusable components inside a suitable /components folder.

The VehicleCard component must accept the following props: model, type, pricePerDay, available.

Display all four values.

If available is false, render a "Not Available" badge instead of a "Book Now" button; if true, render the "Book Now" button.

Use props to pass vehicle data from the parent component to VehicleCard.

Detailed UI styling is not required. Focus on component structure, composition, and correct use of props.

> **Hint:** a simple ternary inside the component handles the button/badge swap cleanly.

4 Marks

Task 2: React Routing and State Management

Configure React Router with the following routes:

Route	Component
/	LoginPage
/fleet	FleetPage (protected)
/my-bookings	MyBookingsPage (protected)
/admin	AdminPanel (lazy-loaded)

Create a navigation component containing links to all routes.

Navigation must be performed using React Router links without a full-page reload.

In FleetPage, create a booking form containing at least: selected vehicle, pickup date, return date.

Use useState to manage the form data.

At least two state values must be used meaningfully. For example: selected vehicle, pickup date.

Display the selected vehicle or another chosen value on the page as the state changes.

Implement an AuthContext holding { customer, token }. A ProtectedRoute wrapper must redirect unauthenticated users to /.

> **Hint:** <Navigate to="/" /> for the redirect; keep the selected vehicle in one piece of state.

4 Marks

Task 3: Express REST API + Middleware

Build the Express backend for managing vehicle bookings.

Implement the following REST endpoints:

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Authenticate customer, issue token
GET	/api/v1/vehicles	Return all vehicles (public)
POST	/api/v1/bookings	Create a new booking (protected)
GET	/api/v1/bookings/my	Return the customer's own bookings (protected)
PATCH	/api/v1/bookings/:id/status	Update booking status (protected)

Create a custom requestLogger middleware that logs [METHOD] [PATH] [TIMESTAMP] for every request, applied globally.

Create an authGuard middleware that validates the Bearer token and returns 401 if missing or invalid, applied to every route except /auth/login and the public vehicle listing.

Implement a global error-handling middleware as the last middleware, returning structured JSON instead of a raw error stack.

Use appropriate HTTP status codes: 200, 201, 400, 401, 500 as applicable. Test the APIs using Postman or Thunder Client.

> **Hint:** keep the auth check and the booking-creation logic in separate functions so each is easy to explain in viva.

4 Marks

Task 4: REST API Consumption in React

Use the Express API developed in Task 3 from the React frontend.

In FleetPage, retrieve vehicle information using: GET /api/v1/vehicles.

Use fetch or Axios.

Use useEffect() so that the API request is made when the component mounts.

Maintain three states: vehicles, loading, and error.

The page must: 1) Display a loading indicator while the request is in progress. 2) Display a friendly error message if the request fails. 3) Render a grid of VehicleCard components after a successful request.

The vehicle information must be rendered from the API response and must not be hardcoded in the React component.

Add a client-side type filter (All | Hatchback | Sedan | SUV) applied to the already-fetched array, without triggering a new API request.

Hint: keep the full array in state; derive the filtered list with .filter().

4 Marks

Task 5: MongoDB + Mongoose Schema Design and Validation

Create the Mongoose-based implementation for the DriveEasy database. Connect to MongoDB using Mongoose.

Create the following three schemas.

Schema	Field Requirement
Customer	name required; email required, unique; licenseNumber required, unique
Vehicle	model required; type required; pricePerDay min 0; available Boolean, default true
Booking	customerId ref Customer; vehicleId ref Vehicle; pickupDate required; returnDate required; status enum, default 'reserved'

The Booking status field must use: reserved, ongoing, completed, cancelled.

Use Mongoose references for: customerId → Customer, vehicleId → Vehicle.

POST /api/v1/bookings must check that vehicle.available === true before creating a booking (return 400 if not), create the Booking record, and then set the vehicle's available field to false.

GET /api/v1/bookings/my must use .populate('customerId', 'name email') and .populate('vehicleId', 'model type').

Connect MongoDB using a connection string stored in .env. For example: MONGO_URI=your_mongodb_connection_string.

Demonstrate at least one validation failure, such as booking an already-unavailable vehicle, and return a meaningful JSON error response instead of exposing the raw Mongoose error object.

Hint: a sequential findById check followed by findByIdAndUpdate is acceptable; a Mongoose transaction is a bonus, not a requirement.

Submission Requirements

Public GitHub repo itue301-exam-[roll-number]-[batch] with /frontend, /backend, README, .env.example. PDF [RollNo]_SetE_Report.pdf with screenshots: (1) FleetPage with vehicle cards, (2) Postman showing 201 on booking creation, (3) MongoDB showing a saved document.

Final Submission (both required): ① GitHub repository link + final commit SHA · ② [RollNo]_SetA_Report.pdf: Upload pdf to Google Classroom / Microsoft Team Assignment before the exam ends.